

ভারতীয় প্রযুক্তিবিদ্যা প্রতিষ্ঠান খড়্গপুর
भारतीय प्रौद्योगिकी संस्थान खड़गपुर
INDIAN INSTITUTE OF TECHNOLOGY KHARAGPUR

Agentic AI

Concepts, Architectures, Cloud Infrastructure, and Research

Dr. Animesh Chaturvedi

Assistant Professor: **IIT Kharagpur**

Young Researcher: **Heidelberg Laureate Forum**

and **Pingala Interaction in Computing**

Young Scientist: **Lindau Nobel Laureate Meetings**

Postdoc: **King's College London** & **The Alan Turing Institute**

PhD: **IIT Indore** MTech: **IIITDM Jabalpur**

Introduction and Foundations

The Central Question

What happens when AI moves from answering questions to pursuing goals?

Generative AI

A user provides a prompt and the model generates a response, typically within a single interaction.

Agentic AI

A user provides an objective and the system can decide what information, reasoning, tools, and actions are required to accomplish that objective.

The fundamental transition is from response generation to goal-directed action.

Evolution of Artificial Intelligence

1. **Traditional AI:** explicit rules and predefined logic.
2. **Machine Learning:** patterns learned from data for prediction and decision support.
3. **Deep Learning:** learned representations for complex data.
4. **Generative AI:** creation of text, images, code, and other content.
5. **Agentic AI:** goal-directed systems that combine models with tools, actions, and feedback.

Interpretation

These paradigms are not strict replacements for one another. Agentic systems build on existing AI capabilities and add orchestration, tool use, and feedback-driven action.

Traditional AI vs Generative AI vs Agentic AI

Paradigm	Primary behaviour	Typical interaction
Traditional AI	Applies predefined rules or learned decision functions.	Input → prediction/decision
Machine Learning	Learns patterns from data to make predictions.	Data → model → prediction
Generative AI	Generates new content from a prompt or context.	Prompt → generated content
Agentic AI	Pursues an objective using reasoning, tools, actions, and feedback.	Goal → plan → act → observe → adapt

Agentic AI adds an action-oriented control loop around AI models.

What Is an AI Agent?

An *AI agent* is a software system that perceives information about its environment, reasons about a goal, selects actions, uses available tools, observes the consequences, and iteratively adapts its behaviour.

A useful abstraction is:

Goal + State + Reasoning + Tools + Action + Feedback

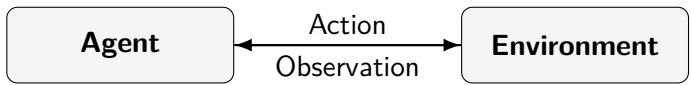
An LLM can be the reasoning component of an agent, but an LLM by itself is not necessarily an agent.

How Agents Work

A Rational Agent Perspective

A classical AI agent can be viewed as a system that:

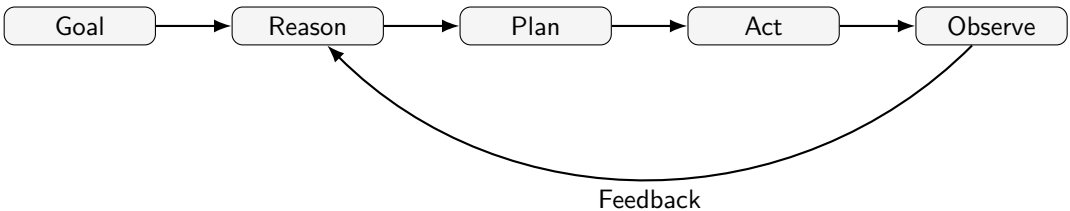
- receives observations from an environment,
- maintains information about the current state,
- chooses an action,
- receives new observations,
- and continues until a task or objective is achieved.



Agentic AI extends this classical idea by using foundation models, external tools, memory, planning mechanisms, and modern software infrastructure.

The Agentic AI Loop

Agentic AI refers to AI systems that can reason about a goal, plan actions, invoke tools, observe results, and revise their actions until a task is completed or requires human intervention.



Goal → Reason → Plan → Act → Observe → Re-plan

Why the Feedback Loop Matters

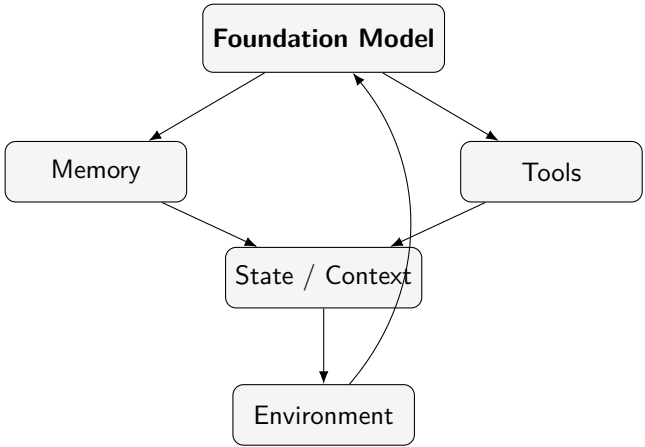
Consider an agent asked to analyse a company's sales performance.

1. The agent identifies the required data.
2. It retrieves the data from a database.
3. It performs analysis.
4. It discovers that one required variable is missing.
5. It changes its plan and retrieves another data source.
6. It continues the analysis.

Key Difference

A fixed workflow follows a predefined sequence. An agent can potentially modify its next action based on what happened in the previous action.

Core Components of an Agent



Perception: What Does the Agent Know?

An agent may obtain information from many sources:

- Natural-language instructions from a human.
- Documents and knowledge bases.
- Databases and structured enterprise data.
- Web pages and external APIs.
- Images, audio, video, and other modalities.
- System logs, sensors, or application state.

Perception is the process of transforming environmental information into a representation that the agent can reason about.

Reasoning and Decision Making

The agent must answer questions such as:

- What is the actual objective?
- What information is currently available?
- What information is missing?
- What action should be performed next?
- Which tool is appropriate?
- Did the previous action produce the expected result?
- Should the current plan be changed?

Reasoning converts observations and goals into decisions.

Planning: From Goal to Subtasks

Suppose the goal is:

"Prepare a market-entry report for an AI product."

The agent may decompose it into:

1. Understand the target market.
2. Identify major competitors.
3. Collect market and competitor data.
4. Analyse customer segments.
5. Estimate opportunities and risks.
6. Develop strategic recommendations.
7. Produce the final report.

Planning transforms a high-level objective into a sequence or hierarchy of executable subtasks.

Planning Strategies

Sequential planning

$$Task \rightarrow T_1 \rightarrow T_2 \rightarrow T_3 \rightarrow Result$$

Hierarchical planning

$$Goal \rightarrow Subgoals \rightarrow Tasks \rightarrow Actions$$

Dynamic planning

$$Plan \rightarrow Action \rightarrow Observation \rightarrow Re-plan \rightarrow Action$$

Dynamic planning is particularly useful when the environment is uncertain or when the result of one action changes what should happen next.

Memory in Agentic AI

Memory allows an agent to maintain useful information beyond the immediate model output.

- **Working memory:** information required for the current task.
- **Long-term memory:** persistent information retained across tasks.
- **Episodic memory:** records of previous experiences or interactions.
- **Semantic memory:** facts and structured knowledge.

Memory allows an agent to behave consistently across multiple steps and, in some systems, across multiple sessions.

Tools: Giving AI the Ability to Act

An LLM normally produces text.
An agent can use tools such as:

- Web search
- Databases
- Python
- Calculators
- APIs
- Code repositories
- Cloud services
- Email
- Calendars
- Enterprise applications

Tools connect the reasoning capability of an AI model to the external world.

Tool Calling / Function Calling

The model does not need to perform every operation itself.
Instead, it can decide:

"I need to invoke this tool with these parameters."

Example:

1. User: "Find tomorrow's available meeting slots."
2. Agent determines that calendar information is required.
3. Agent calls the calendar tool.
4. Calendar returns available slots.
5. Agent interprets the result and responds.

Tool calling separates language-based decision making from the actual execution of external operations.

RAG and Agentic RAG

Retrieval-Augmented Generation (RAG)

$$\textit{Query} \rightarrow \textit{Retrieve} \rightarrow \textit{Generate}$$

RAG improves an LLM's access to external information.

Agentic RAG

$$\textit{Goal} \rightarrow \textit{Determine Information} \rightarrow \textit{Search} \rightarrow \textit{Evaluate} \rightarrow \textit{Search Again} \rightarrow \textit{Generate}$$

RAG provides retrieval.
Agentic systems can make retrieval itself part of a broader goal-directed decision process.

The ReAct Concept

A widely discussed agent pattern interleaves reasoning and action:

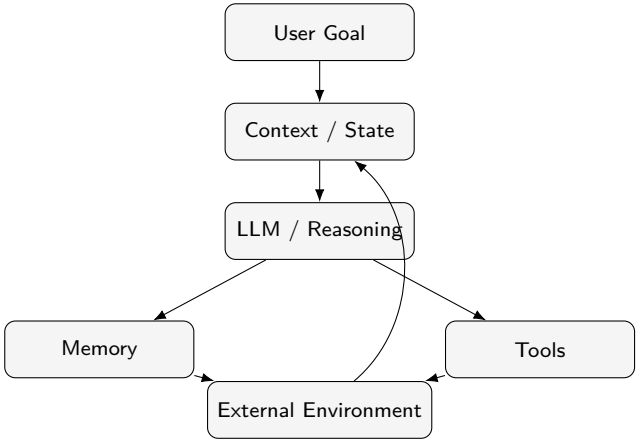
$$\textbf{Reason} \rightarrow \textbf{Act} \rightarrow \textbf{Observe}$$

Example:

- The agent identifies that current information is insufficient.
- It chooses a search tool.
- The search result becomes a new observation.
- The agent determines the next required action.
- The cycle continues until sufficient evidence is available.

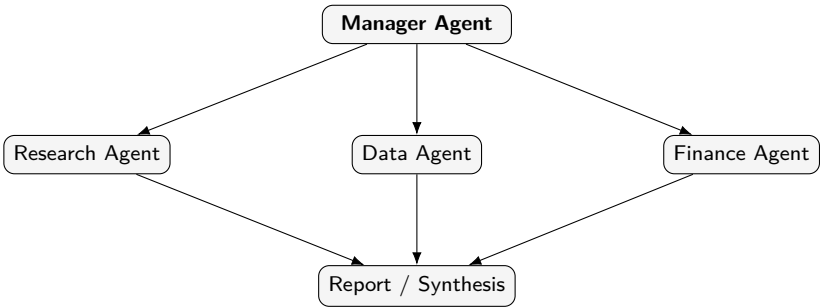
The important idea is not a particular prompting format; it is the repeated interaction between decision making and environmental feedback.

A Complete Single-Agent Architecture



From One Agent to Multiple Agents

A complex problem can be decomposed among specialised agents.



Architectures and Coordination

Why Use Multi-Agent Systems?

Different agents can specialize in different responsibilities.

- **Specialization:** each agent can focus on a particular task or domain.
- **Parallelism:** independent tasks can potentially be executed simultaneously.
- **Modularity:** individual agents can be modified without redesigning the entire system.
- **Debate:** multiple agents can produce and critique alternative solutions.
- **Coordination:** a manager agent can combine outputs from specialized agents.

Multi-agent systems introduce a new problem: the system must coordinate not only actions, but also the agents that decide those actions.

Agent Coordination

Multi-agent systems can involve:

- **Collaboration:** agents pursue a shared objective.
- **Delegation:** one agent assigns subtasks to another.
- **Negotiation:** agents resolve conflicting preferences or resources.
- **Competition:** agents generate competing solutions.
- **Consensus:** agents combine multiple perspectives into a final decision.

This connects Agentic AI with distributed AI, multi-agent systems, game theory, and organizational decision making.

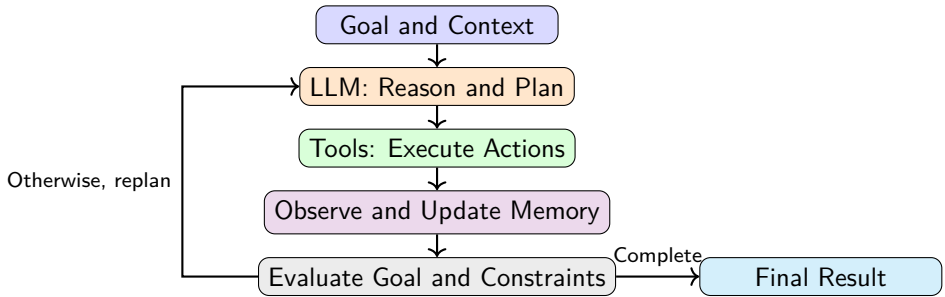
Agentic AI vs Reinforcement Learning

These concepts are related but not identical.

Reinforcement Learning	**LLM-based Agent**
State $\rightarrow$ Action $\rightarrow$ Reward	Goal $\rightarrow$ Reason $\rightarrow$ Action $\rightarrow$ Observation
Policy is often learned from interaction.	Decision making may be driven by a foundation model and orchestration logic.
Reward provides a central learning signal.	Success criteria can include task completion, safety, quality, and human feedback.

An LLM agent does not have to be trained using reinforcement learning.

How the Technologies Work Together



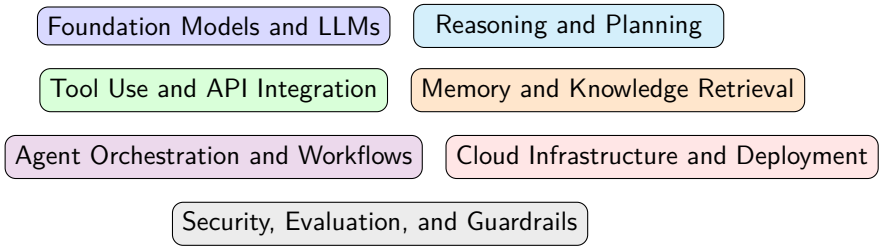
Takeaway

Agentic behavior emerges from the interaction of models, planning, tools, memory, feedback, and execution controls—not from the LLM alone.

Enabling Technologies: Embeddings and Integration

Key Enabling Technologies Behind Agentic AI

Agentic AI combines multiple technologies to enable AI systems to understand goals, make decisions, execute actions, and adapt based on the observed results.



Core Idea

An LLM provides language understanding and generation, while the surrounding technologies enable an agent to plan, use tools, retain relevant information, and perform tasks reliably.

What Is an Embedding in an LLM?

An embedding represents a token, sentence, document, or other input as a numerical vector that a machine-learning model can process.

Example:

```
"Search research papers on software evolution"
```

An embedding model transforms the input into a vector:

$$\mathbf{x} = \begin{bmatrix} 0.21 & -0.34 & 0.78 & 0.15 & \cdots \end{bmatrix}$$

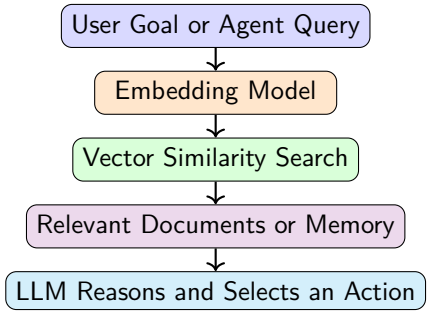
- Each number is a component of the embedding vector.
- The vector has a fixed number of dimensions for a given model.
- The dimensions collectively encode learned patterns of meaning and other useful features.

What Does Embedding Dimension Mean?

If an embedding contains d numerical components, its dimension is d. For example, a vector with 768 components has 768 dimensions.

How Embeddings Enable Agentic AI

Embeddings help agents retrieve relevant knowledge and information from large collections of documents, memories, and tool descriptions.



Key Point

Embeddings support retrieval and matching; the agent's reasoning and control logic determine what to do with the retrieved information.

Why Does Agentic AI Need Cloud Services?

Consider an agent asked to:

"Analyze recent research papers and prepare a literature review."

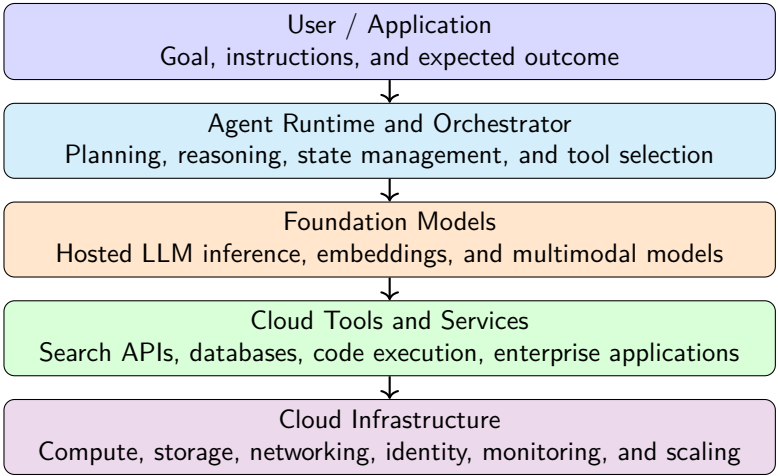
The agent may need to:

- Use an LLM to understand the task and generate a plan.
- Search the web or query a research-paper database.
- Store retrieved documents and intermediate results.
- Execute code to analyze and summarize information.
- Maintain task state and monitor execution.

Role of the Cloud

Cloud services provide on-demand computing, hosted AI models, storage, databases, APIs, orchestration, and security infrastructure that enable agents to operate reliably at scale.

Cloud Architecture for an AI Agent



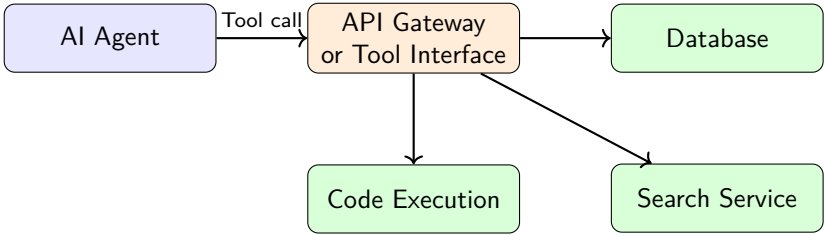
The agent decides what action to take; cloud services provide the capabilities to execute that action.

Cloud Service Categories Used by AI Agents

Cloud capability	Purpose	Example services
AI model hosting	Generate responses, plans, and tool calls	Amazon Bedrock, Azure AI Foundry, Vertex AI
Compute	Run agent code and background tasks	Virtual machines, containers, serverless functions
Storage and Databases	Store documents, files, and task artifacts, Store task state, conversation history, and retrieved knowledge	Object storage and managed file storage, Managed SQL/NoSQL databases, vector search
Integration	Connect agents to external tools and applications	API gateways, queues, workflow services
Security and observability	Control access and trace agent execution	IAM, secrets management, logs, metrics, tracing

Tools and APIs: Connecting Agents to the Cloud

An agent becomes useful when it can interact with real systems.



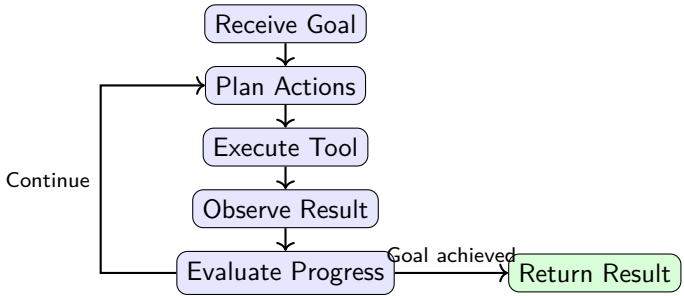
Example tool calls:

- `search_papers(query)`
- `get_customer_record(customer_id)`
- `run_analysis(code, dataset)`
- `create_ticket(description)`

Tools should enforce authentication, authorization, input validation, and appropriate limits on agent actions.

Orchestration: Managing Multi-Step Agent Workflows

Complex tasks may require multiple actions and repeated decisions.



Cloud orchestration services can help with:

- Scheduling tasks and coordinating services.
- Handling retries, timeouts, and failures.
- Running independent subtasks in parallel.
- Persisting workflow state and tracking execution.

Example: A Cloud-Based Research Assistant Agent

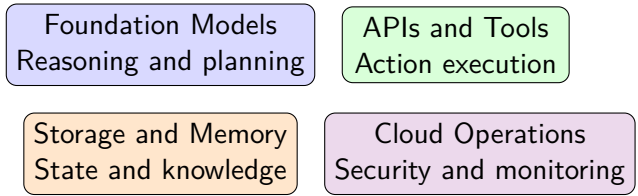
User goal: "Prepare a literature review on Graph Neural Networks for Software Evolution Analysis."

1. **Planning:** The LLM creates search and synthesis subtasks.
2. **Retrieval:** Search APIs and scholarly databases locate papers.
3. **Storage:** Documents and metadata are saved in cloud storage.
4. **Knowledge extraction:** An embedding model supports semantic retrieval.
5. **Analysis:** The agent compares methods, datasets, and findings.
6. **Validation:** It checks citations and flags missing evidence.
7. **Output:** The agent generates a structured literature review.

Possible Cloud Components

Hosted LLM + agent runtime + search APIs + object storage + database/vector search + workflow orchestration + monitoring.
These components can be selected independently; a single cloud provider or a managed agent product is not mandatory.

Cloud Services Enable Agentic AI



- Cloud platforms supply scalable infrastructure and managed AI services.
- Agent runtimes coordinate model calls, tools, memory, and task execution.
- APIs connect agents to enterprise data and external applications.
- Security, observability, and cost controls are essential for production.

Final Takeaway

LLM + Tools + Memory + Orchestration + Cloud Infrastructure + Governance = A Deployable Agentic AI System

Applications and Case Study

Business Applications of Agentic AI

Customer Service

- Understand customer request
- Retrieve account information
- Apply policies
- Execute approved actions

Marketing

- Analyse campaigns
- Identify customer segments
- Generate content
- Monitor outcomes

Finance

- Analyse reports
- Detect anomalies
- Forecast trends
- Prepare management reports

Supply Chain

- Monitor inventory
- Analyse demand
- Identify disruptions
- Recommend corrective actions

Agentic AI in Data Science

A data science agent could transform:

Business Question

into:

$$\textit{DataDiscovery} \rightarrow \textit{Cleaning} \rightarrow \textit{EDA} \rightarrow \textit{FeatureEngineering} \rightarrow \textit{Modeling}$$

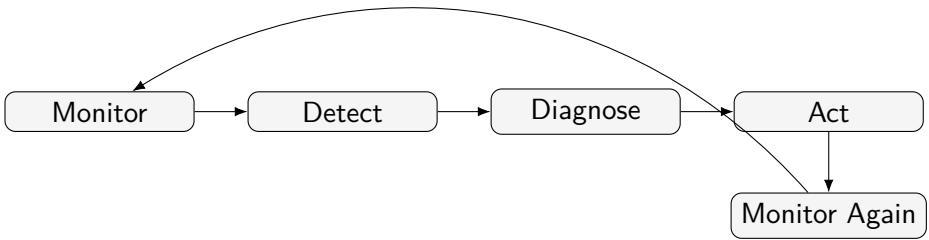
$$\textit{Modeling} \rightarrow \textit{Evaluation} \rightarrow \textit{Interpretation} \rightarrow \textit{Report}$$

The agent may select tools dynamically depending on intermediate results.

The critical research question is not whether an agent can execute these steps, but whether it can reliably determine that its analysis and conclusions are correct.

Agentic AI and Cloud Computing

An agent can operate as an intelligent control layer over cloud infrastructure.



Applications include autoscaling, fault management, resource optimization, cost optimization, and AIOps.

Case Study: Market Entry Strategy

Goal: Develop a market-entry strategy for an EV company.

An agentic system could:

1. Research the target market.
2. Identify competitors.
3. Retrieve government and industry information.
4. Analyse customer segments.
5. Estimate market opportunities.
6. Analyse competitors' pricing.
7. Develop alternative strategies.
8. Compare risks and expected outcomes.
9. Produce an executive report.

The agent is not simply writing the report; it is orchestrating the work required to produce the report.

Case Study: Market Entry Strategy

Where Can It Fail?
To identify possible failure points.

- What if the market data is outdated?
- What if a search result contains misleading information?
- What if the agent misunderstands the business objective?
- What if two agents provide contradictory conclusions?
- What if the financial assumptions are incorrect?
- What if the agent takes an unauthorized action?
- Who should approve the final strategic recommendation?

Capability is only one dimension; reliability and governance are equally important.

Reliability, Safety, and Evaluation

Human-in-the-Loop

Autonomy should be proportional to the consequences of an action.

Risk level	Recommended control
Low	Automate routine, reversible actions.
Medium	Execute with monitoring and clear escalation rules.
High	Require explicit human approval before execution.

High-risk examples include financial transactions, deletion of important data, production infrastructure changes, and decisions affecting individuals.

Major Challenge: Hallucination

A conventional hallucination produces:

Incorrect information

An agentic hallucination can produce:

$$\text{Incorrect belief} \rightarrow \text{Incorrect plan} \rightarrow \text{Incorrect action} \rightarrow \text{Real-world consequence}$$

Agentic systems amplify the consequences of unreliable reasoning because their outputs can trigger actions.

Security Challenges

Giving an agent access to tools creates a larger attack surface.
Important risks include:

- **Prompt injection:** malicious instructions influence the agent's behaviour.
- **Data leakage:** sensitive information may be exposed to inappropriate tools or users.
- **Unauthorized actions:** the agent may invoke tools beyond intended permissions.
- **Credential exposure:** tools may provide access to sensitive credentials or services.
- **Excessive autonomy:** an error can become an operational incident.

Agent security must protect the entire action chain, not only the language model.

Cascading Errors

Agentic systems can create chains of dependent decisions.

$$E_1 \rightarrow D_1 \rightarrow A_1 \rightarrow E_2 \rightarrow D_2 \rightarrow A_2 \rightarrow \cdots$$

where:

- E represents an observation or environmental input,
- D represents a decision,
- A represents an action.

A small error early in an autonomous workflow can propagate through later decisions, making error detection and recovery essential.

How Do We Evaluate an Agent?

Traditional ML metrics are not sufficient for many agentic tasks.
Important evaluation dimensions include:

- **Task success rate:** Did the agent actually achieve the objective?
- **Accuracy and factuality:** Were the intermediate and final outputs correct?
- **Efficiency:** How many steps and tool calls were required?
- **Cost:** How much computation, token usage, and API usage were consumed?
- **Latency:** How long did the complete task require?
- **Robustness:** Can the agent recover from failures?
- **Safety:** Did the agent avoid unauthorized or harmful actions?

Research Directions and Synthesis

Research Opportunities

Agentic AI creates research opportunities across several disciplines.

AI / ML

- Planning
- Reasoning
- Memory
- Agent learning

Data Science

- Autonomous analytics
- Data quality
- Experimentation
- Evaluation

Systems

- Agent orchestration
- Cloud resource management
- Agent observability
- Cost optimization

Software Engineering

- Autonomous debugging
- Software evolution
- Code agents
- Repository intelligence

Agentic AI for Indian-Language and Social Applications

Agentic AI can potentially combine:

- Multilingual language models.
- Retrieval from local-language knowledge sources.
- Translation and cross-lingual reasoning.
- Government and public-service APIs.
- Regional knowledge bases.
- Human-in-the-loop validation.

Potential applications include:

Education **Healthcare Information** **Public Services** **Agriculture** **Local Commerce**

The major research challenge is not only multilingual generation, but reliable goal-directed action across heterogeneous and culturally diverse environments.

Agentic AI in Software Engineering

Consider the objective:

"Find and fix a bug in our software repository."

An agent could potentially:

1. Inspect the repository structure.
2. Search for relevant code and dependencies.
3. Analyse the suspected defect.
4. Generate a candidate patch.
5. Execute tests.
6. Analyse failing tests.
7. Modify the patch.
8. Run regression tests.
9. Prepare a pull request for human review.

The agent operates over a changing software environment rather than merely generating code.

GitHub Copilot as Agentic AI

- Integrated into VS Code to assist software development.
- Goes beyond code completion through multi-step task execution.
- Can inspect files, edit code, run commands, and analyze test results.
- Uses an iterative loop: Plan $\rightarrow$ Act $\rightarrow$ Observe $\rightarrow$ Revise.

Example: Agent Mode in Action Task: Add input validation to a registration API.

1. Inspect relevant source files.
2. Plan the required code changes.
3. Modify implementation and create unit tests.
4. Run tests and analyze failures.
5. Revise changes and summarize the result.

Key idea: Copilot coordinates multiple actions toward a goal.

Agentic AI: A Systems Research View

Agentic AI can be understood as a combination of:

$$\boxed{\text{Foundation Models} + \text{Planning} + \text{Memory} + \text{Tools} + \text{Environment} + \text{Feedback}}$$

This creates a new systems problem:

How do we build agents that are capable, reliable, efficient, secure, and controllable?

This question is likely to require contributions from:

$$\text{AI} + \text{Data Science} + \text{Software Engineering} + \text{Distributed Systems} + \text{Management}$$

Key Takeaways

1. An **LLM is a model**; an **agent is a larger system**.
2. Agentic AI focuses on **goal-directed behaviour**.
3. Agents combine reasoning with **planning, memory, tools, actions, and feedback**.
4. The central loop is:

$$\textbf{Goal} \rightarrow \textbf{Reason} \rightarrow \textbf{Plan} \rightarrow \textbf{Act} \rightarrow \textbf{Observe} \rightarrow \textbf{Adapt}$$

5. Multi-agent systems distribute complex work across specialized agents.
6. Greater autonomy creates greater opportunities as well as greater risks.
7. Evaluation must measure not only response quality but also task success, efficiency, robustness, and safety.
8. Developer's Role: Review code changes, verify test results, and approve sensitive actions.
9. Agentic AI supports development; it does not replace developer judgment.
10. An agent is not just an LLM. It is a software system that combines a model with tools, state, execution logic, and operational controls.


תודה רבה
Hebrew
Ευχαριστώ
Greek
Danke
German
Спасибо
Russian
धन्यवादः
Sanskrit
Merci
French
ধন্যবাদ
Bangla
நன்றி
Tamil
شكراً
Arabic
ಧನ್ಯವಾದಗಳು
Kannada
Thank You
English
നന്ദി
Malayalam
Grazie
Italian
ధన్యవాదాలు
Telugu
આભાર
Gujarati
多謝
Traditional Chinese
ਧੰਨਵਾਦ
Punjabi
धन्यवाद
Hindi & Marathi
Gracias
Spanish
多谢
Simplified Chinese
https://sites.google.com/site/animeshchaturvedi07
Obrigado
Portuguese
ありがとうございました
Japanese
ขอบคุณ
Thai
감사합니다
Korean